INTEGRAL DRAGON

AI-ACCELERATED SOFTWARE DEVELOPMENT • FOUNDATIONS

Foundations Overview

Modules 1 through 4 — the operating baseline for trustworthy AI-accelerated delivery.

AI-Accelerated Software Development | Foundations Program

Confidential. Prepared exclusively for Ensemble Health Partners.

Foundations is not a theory block — it is the operating baseline

- Replace ad hoc AI usage with a disciplined engineering operating model
- Establish the CLI-native working environment before acceleration features are added
- Teach control surfaces — specs, plans, rules, permissions, review — before scope expands
- Prepare learners to enter Labs with repeatable habits, not one-off prompt tricks

The goal is not faster prompts.
The goal is controlled, reviewable, enterprise-ready engineering work.

The four-module arc: each layer adds control before adding power

MODULE 1

The Agentic Paradigm and CLI Environment

Sessions 1–4

Adopt the agentic posture and understand the tool.

MODULE 2

Spec-Driven Development and Context Control

Sessions 5–8

Constrain intent, execution, and context.

MODULE 3

Tooling, MCP, and Agentic Orchestration

Sessions 9–12

Extend capability and delegate work safely.

MODULE 4

Enterprise Security, Governance, and Auditability

Sessions 13–16

Secure, govern, and produce auditable evidence.

→ Each module assumes the previous one is in place. Control before capability is the sequence.

The Agentic Paradigm and CLI Environment

Session 1: The Agentic Engineering Shift

Replace casual AI usage with a structured model: output, reviewability, and ownership all matter simultaneously.

Session 2: LLM Internals for Orchestrators

Build mechanical sympathy — tokens, inference, variance, context windows — to make better workflow decisions.

Session 3: Context Boundaries and Rot

Understand why context quality matters more than context volume, and how to apply the minimum working set principle.

Session 4: Claude Code CLI Architecture and Setup

Install and correctly bound the harness — context init, tool injection, trust boundary, approval mode.

Core message: AI writes syntax; engineers still own architecture, judgment, and liability.

Spec-Driven Development and Context Control

Session 5: Spec-Driven Development

Define machine-readable intent — goal, constraints, acceptance criteria, non-goals — before generation begins.

Session 6: Plan Mode vs. Act Mode

Separate codebase analysis from mutation. Review the plan before approving execution.

Session 7: Context Control via CLAUDE.md

Persist project rules so constraints survive across sessions, engineers, and agents without re-prompting.

Session 8: Extending Capabilities with MCP

Connect external systems through governed tool boundaries — security, payload legibility, and authorization.

Core message: Reliable agentic engineering starts with explicit intent and governed context.

Tooling, MCP, and Agentic Orchestration

Session 9: Skills and Deterministic Hooks

Decide what remains flexible (skills) versus what becomes automatic enforcement (hooks).

Session 10: Task Decomposition and Subagents

Break large work into bounded delegated units. Give each subagent a clear contract and scope.

Session 11: Parallel Orchestration: Agent Teams

Coordinate multiple agents without losing state discipline or collision control.

Session 12: OWASP Top 10 for Agentic Apps

Secure the broader execution surface created by tooling, teams, and expanded capability.

Core message: Once the workflow is constrained, capability can expand safely.

Enterprise Security, Governance, and Auditability

Session 13: Zero Trust and Permission Boundaries

Apply least privilege to agent execution — trust no implicit scope, verify all access.

Session 14: Secure MCP Governance

Govern external tool and MCP trust boundaries — authorization, payload inspection, blast radius.

Session 15: Code Review and Cognitive Debt Mitigation

Review AI-generated diffs for structural risk and comprehension debt, not just surface correctness.

Session 16: Auditability and Workflow Narration

Preserve enough evidence to reconstruct why a change happened — narration, traceability, compliance.

Core message: Enterprise use is not real until permissions, review, and traceability hold under scrutiny.

The control stack — from posture to auditable evidence

1
Posture & Mental Model
Output · Reviewability · Ownership · Mechanical sympathy

2
Bounded Context & Explicit Intent
Specs · Plan Mode · CLAUDE.md · Minimum working set

3
Deterministic Enforcement & Decomposition
Hooks · Subagents · Agent teams · OWASP threat model

4
Permissions, Review & Auditable Evidence
Zero trust · MCP governance · Cognitive debt · Narration

- Each layer depends on the previous. Skipping the posture layer makes the permission layer theater.
- This is the path from AI assistance to enterprise-grade agentic engineering.

Session progression at a glance

S01–S04	Posture, mechanics, context quality, and CLI harness setup. The mental model and the environment before any tooling is added.
S05–S08	Specs, plans, persistent project rules, and governed external tools. Intent becomes explicit; context becomes bounded.
S09–S12	Hooks, subagents, parallel agent teams, and the OWASP threat surface. Capability expands inside the constraints already in place.
S13–S16	Zero trust permissions, MCP governance, cognitive debt review, and auditable narration. Enterprise-grade delivery requires all of these simultaneously.

→ **Each module adds control before adding more power.** The sequence is intentional.

What learners can do by the end of Foundations

- ✓ Explain the difference between vibe coding and structured agentic engineering
- ✓ Drive work from specs and reviewed plans rather than open-ended prompts
- ✓ Decompose work across tools and agents with clear delegation contracts
- ✓ Choose the smallest useful context for a task and defend that choice
- ✓ Operate Claude Code inside explicit permission and context boundaries
- ✓ Review and narrate AI-assisted changes in terms a team can inherit and audit

> Labs assume these capabilities are already in place. Learners who enter Labs without this foundation will be working at a disadvantage from the first exercise.

Labs assume Foundations is already in practice — not just understood

- Labs assume the learner can already work in the CLI without guidance on setup or basics
- Labs assume bounded scope and review discipline are the learner's default posture
- Labs assume AI output is treated as untrusted until verified by tests or review
- Labs build on this foundation with codebase reconnaissance, test generation, and CI loops

> The difference between a learner who struggled through Foundations and one who internalized it will be visible in the first lab exercise.

KEY TAKEAWAY

Control before capability. Structure before speed.

Foundations is the operating baseline for trustworthy AI-accelerated delivery. Every session that follows builds on what this program establishes.